

COMP202 Week 2 - solutions

1. Give a linear ($O(n)$) running time algorithm for the maximum-subarray problem. Hint: Try a non-recursive algorithm.

Solution: Let A be the input array. Scan A from left to right. In the j 'th step, store the largest subarray sum that ends (and includes!) the j 'th element.

```
1: Input:  $A = [a_1, \dots, a_n]$ .
2:  $\text{bestsum} \leftarrow -\infty$ 
3:  $\text{currentsum} = 0$ .
4: for  $i = 1$  to  $n$  do
5:    $\text{currentsum} = \max(a_i, \text{currentsum} + a_i)$ 
6:    $\text{bestsum} = \max(\text{bestsum}, \text{currentsum})$ 
7: end for
8: return  $\text{bestsum}$ 
```

Loop invariant: After the i 'th step, currentsum holds the solution to the maximum-subarray problem for input $[a_1, \dots, a_i]$ with the additional restriction that a_i must be included. The variable bestsum holds the solution to the maximum-subarray problem for input $[a_1, \dots, a_i]$.

Proof: This can be proved by induction. The base case is easy. In the j 'th step, the old value of currentsum holds $\max_{i \in \{1, \dots, j-1\}} \{a_i + \dots + a_{j-1}\}$. That is, before the updates on Line 5 and Line 6, currentsum holds the solution to the maximum-subarray problem for $[a_1, \dots, a_{j-1}]$ where a_{j-1} must be included. Thus, after the updates, currentsum equals $\max_{i \in \{1, \dots, j-1\}} \{a_i + \dots + a_j, a_j\}$, which equals the solution to the maximum-subarray problem for input $[a_1, \dots, a_j]$ with the restriction that a_j must be included. Moreover, it can be seen that bestsum stores the value of the best solution seen so far, which is the solution to the maximum-subarray problem for input $[a_1, \dots, a_j]$.

2. Describe a *divide-and-conquer* algorithm to find the **minimum** of a set of n numbers. Your algorithm should clearly describe how the problem is divided into subproblems. Either give some psuedo-code or describe your algorithm clearly in words. Assume that the set is stored in an array $A = [A[1], \dots, A[n]]$.

(Only a divide-and-conquer algorithm will be accepted here. Of course you can find the minimum in time $O(n)$, but the point is to understand the divide-and-conquer paradigm.)

What is the running time of your algorithm?

Solution: I hope that one idea for a solution is pretty clear here. What do you do?

Divide the array A into two (nearly equal) parts. Find the minimum of each part (recursively). So you'll have $\min_\ell$ and $\min_r$ which are the minimums of the “left” and “right” sublists. Then the minimum of the whole list is the smaller of $\min_\ell$ and $\min_r$.

Suppose, for the sake of convenience that n is a power of 2, i.e. $n = 2^k$ for some non-negative integer k .

At level i (in the recursion) you will have a collection of 2^i sets, each of size $n/2^i$. So on the “bottom” level, you'll end up with $n/2$ pairs of numbers. You'll find the minimum of each pair (using a single comparison) to give you a set of $n/2$ numbers as you work your way back up the recursive calls.

Then you'll perform $n/4$ comparisons to end up with $n/4$ numbers, and so forth. As you work your way back up the recursive calls, at level i you will have reduced the list down to a set of $n/2^{k-i}$ numbers, which you have divided into pairs and you will find the minimum of each pair.

In total, how many comparisons will you perform? As I said, if you use this idea, you'll perform $n/2$ comparisons, then $n/4$, then $n/8, \dots$, then finally a single comparison to finish finding the minimum. So the total number of comparisons is

$$\begin{aligned} n/2 + n/4 + \dots + n/n &= 1 + 2 + 4 + 8 + \dots + n/2 \\ &= 1 + 2^1 + 2^2 + 2^3 + \dots + 2^{k-1} \\ &= 2^k - 1 \end{aligned}$$

(using the fact that $\sum_{i=0}^{k-1} c^i = \frac{c^k - 1}{c - 1}$.)

So the total running time of this algorithm is $O(2^k) = O(n)$. The time consuming part of the task will be in constructing the sublists. At each level, constructing the smaller lists will take constant time $O(1)$, because it only need to pass to the recursive calls of the algorithm the start and end index of each smaller subarray. Similar calculation as for the number of comparisons, shows that the total running time is also $O(n)$.

However, observe that we obtain a very precise bound on the total number of comparisons. Namely, the algorithm uses at most $2^k - 1 = n - 1$ comparisons! That is, we do not use here the big-Oh notation, because the constant factor in front of $n - 1$ is one.

3. Use the Master Method to find an asymptotic expression for $T(n)$. (Assume that $T(n) = c$ for $n \leq d$, for some constants $c > 0$ and $d \geq 1$.)

Solution: Note that in all cases here, you want to consider the function $n^{\log_b a}$ and compare this to the function $f(n)$ in a suitable fashion.

(a) $T(n) = 2T(n/2) + \log n$

Here we have that $n^{\log_b a} = n^{\log_2 2} = n$. Recalling that I stated in class that $\log n$ is $O(n^{1/k})$ for all $k \geq 1$, we note that $\log n$ is $O(n^{1/2})$. In other words, there is an $\varepsilon > 0$, for instance $\varepsilon = 1/2$, such that $f(n) = \log n$ is $O(n^{(\log_b a) - \varepsilon}) = O(n^{1 - \varepsilon}) = O(n^{1/2})$.

So we're in Case 1, and $T(n)$ is $\Theta(n)$.

(b) $T(n) = 7T(n/3) + n^2$

First note that $\log_b a = \log_3 7 < 2$ (strictly smaller than 2). Therefore, we can find $\varepsilon > 0$ (very small but positive), such that $f(n) = n^2$ is $\Omega(n^{\log_3 7 + \varepsilon})$. Also, we note that $7 \cdot (n/3)^2 = (7/9)n^2$, with $\delta = 7/9 < 1$ (strictly smaller than 1), we see that we're in Case 3.

Therefore $T(n)$ is $\Theta(n^2)$.

(c) $T(n) = 9T(n/3) + n^3 \log n$

Since $\log_b a = \log_3 9 = 2$, we're comparing $n^{\log_b a} = n^2$ and $f(n) = n^3 \log n$. Obviously, we can again find $\varepsilon > 0$ (like $\varepsilon = 1$, say) so that $n^3 \log n$ is $\Omega(n^{2 + \varepsilon})$. So we seem to be in Case 3.

Look now at $a \cdot f(n/b)$. This is $a f(n/b) = 9 \cdot (n/3)^3 \log(n/3) = (1/3)n^3 \log(n/3) \leq (1/3)n^3 \log n = \delta \cdot f(n)$. (Why is this last inequality true?) We can therefore use $\delta = 1/3 < 1$ in our condition for Case 3.

So $T(n)$ is $\Theta(n^3 \log n)$.

(d) $T(n) = 16T(n/2) + (n \log n)^4$

$n^{\log_b a} = n^{\log_2 16} = n^4$. Since $f(n) = (n \log n)^4 = n^4 \log^4 n$, we are in Case 2 this time with $k = 4$ in the Master Method Theorem. (That is, $n^4 \log^4 n$ is obviously $\Theta(n^4 \log^4 n)$.)

So we conclude that $T(n)$ is $\Theta(n^4 \log^5 n)$. (Note the increase in the exponent from $k = 4$ to $k + 1 = 5$ on the logarithm that comes from the Master Theorem.)

(e) $T(n) = 4T(n/2) + n^2$

Here we're comparing $n^{\log_b a} = n^{\log_2 4} = n^2$ and $f(n) = n^2$. Like the previous example right before this one, we're in Case 2 with $k = 0$ this time, because $f(n) = \Theta(n^{\log_b a} \log^k n) = \Theta(n^{\log_b a}) = \Theta(n^2)$ (note that $\log^k n = \log^0 n = 1$).

This means that $T(n)$ is $\Theta(n^2 \log^{k+1} n) = \Theta(n^2 \log n)$.

(f) $T(n) = 2T(n/2) + 10n$

Since $\log_b a = \log_2 2 = 1$, so we're comparing $n^{\log_b a} = n^1 = n$ and $f(n) = 10n$. So we're again in Case 2 with $k = 0$ in this instance too.

Therefore $T(n)$ is $\Theta(n \log n)$.

(g) $T(n) = 8T(n/2) + 1000n^2$

$\log_b a = \log_2 8 = 3$, so we compare $n^{\log_b a} = n^3$ and $f(n) = 1000n^2$. We're in Case 1 since we can find $0 < \varepsilon \leq 1$ such that $1000n^2$ is $O(n^{3-\varepsilon})$ (we can use here for instance $\varepsilon = 0.9$ or even $\varepsilon = 1$).

This means that $T(n)$ is $\Theta(n^3)$.

(h) $T(n) = 3T(n/3) + \sqrt{n}$

Again we're in Case 1 here since we're comparing $n^{\log_3 3} = n$ and $f(n) = \sqrt{n}$. Now take any $0 < \varepsilon \leq \frac{1}{2}$, and we have that $\sqrt{n}$ is $O(n^{\log_3 3 - \varepsilon} = n^{1-\varepsilon})$.

Therefore, $T(n)$ is $\Theta(n)$.

(i) $T(n) = 2T(n/2) + n \log n$

This satisfies Case 2 with $k = 1$ (why?). So $T(n)$ is $\Theta(n \log^2 n)$.

(j) $T(n) = 3T(n/2) + n^2$

This recurrence satisfies Case 3 with, say, $\delta = \frac{3}{4}$. (Why?)

So $T(n)$ is $\Theta(n^2)$.

(k) $T(n) = T(n/2) + 2^n$

This is also an example of Case 3. Certainly there is $\varepsilon > 0$ such that $f(n) = 2^n$ is $\Omega(n^{(\log_2 1) + \varepsilon}) = \Omega(n^\varepsilon)$ (note that $\log_2 1 = 0$). (What is a value of δ that would work here for the other condition on $f(n) = 2^n$?)

In this case we have that $T(n)$ is $\Theta(2^n)$.